Copyright 2026
No Unauthorized Duplication Permitted

Robot Operating System and McMaster AEV Components

In this lab, you test the operation of Robot Operating System (ROS) and McMaster AEV Components. **You are expected to work on some of the objectives given in this lab manual before your lab session and present the results in the lab.** Please follow the following safety procedures for this experiment.

Please note that when you copy and paste commands, syntax error might occur. It is strongly recommended that you manually type the codes instead of copying and pasting them from this document or from the sources on the internet.

Important Safety Procedures to Follow

- Please be careful while you are working with the vehicle.
- In this experiment, you should not operate the Electronic Speed Controller and run the vehicle.
- You are expected to operate Jetson Nano from a separate power supply in this experiment without using the battery.
- If the battery needs to be connected to energize auxiliary devices in the vehicle (other than the electronic speed controller), **the wheels of the McMaster AEV should be elevated or removed so that the vehicle should not move.** The experiments must be conducted in a laboratory environment under the supervision of teaching assistants (TAs). Please follow TA's instructions carefully.
- In this lab, you are using sensitive electronic devices. Please be cautious when you are working with the devices.
- There must be no loose wires or metal pieces on your test bench.
- You must keep terminals and connections out of the way from accidental touching and from contact to test equipment or parts connected to other voltage levels.
- **You must not touch the vehicle or any component of the vehicle if the battery is connected to the vehicle and powers the auxiliary devices (other than VESC).**
- No food or drinks are allowed in the lab.
- If you encounter any issues with the motor drive operation, you must always check with TAs.

Copyright 2026
No Unauthorized Duplication Permitted

Objective 1: Introduction to Robot Operation System

What is Robot Operating System?

Robot Operating System (ROS) is a Berkely Software Distribution (BSD) licensed system for controlling robotic components from a personal computer (PC). A ROS system is comprised of several independent nodes, each of which communicates with the other nodes using a publish/subscribe messaging model. For example, a particular sensor's driver might be implemented as a node, which publishes sensor data in a stream of messages. These messages could be consumed by any number of other nodes, including filters, loggers, and higher-level systems such as guidance and pathfinding.

Nodes in ROS do not have to be on the same system (multiple computers) or even of the same architecture. You could have an Arduino publishing message, a laptop subscribing to them, and an Android phone driving a motor. This makes ROS flexible and adaptable to the needs of the user. ROS is also open source, maintained by many people.

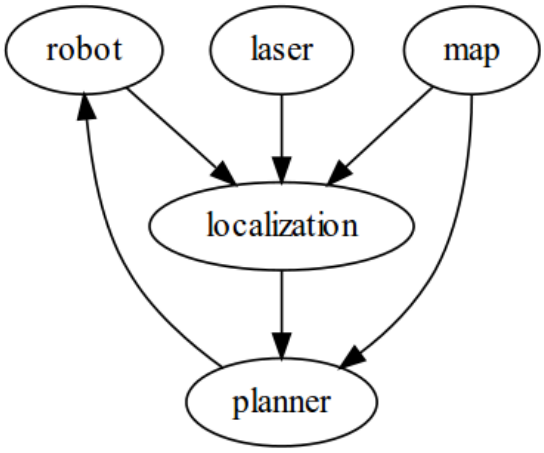

Figure 1: Block diagram for a ROS system

A ROS system can be visualized as a graph where all the vertices are **nodes** and the lines between them are known as **topics**. In simple terms, nodes are programs that perform a particular task and send/receive data in the form of messages. These messages are exchanged between two nodes over

Copyright 2026
No Unauthorized Duplication Permitted

the topic (line) connecting them. An example of a typical ROS system is given in the Figure 1 wherein *robot*, *laser*, *map*, *localisation*, and *planner* are the **nodes**. The arrows connecting the nodes are the **topics**. An arrow from A to B indicates that the topic carries messages from node A to node B.

ROS starts with the ROS Master. The Master allows all other ROS pieces of software (nodes) to find and talk to each other, as shown in Figure 2. That way, we do not have to specifically state "send this sensor data to that computer at 127.0.0.1". We can simply tell Node 1 to send messages to Node 2.

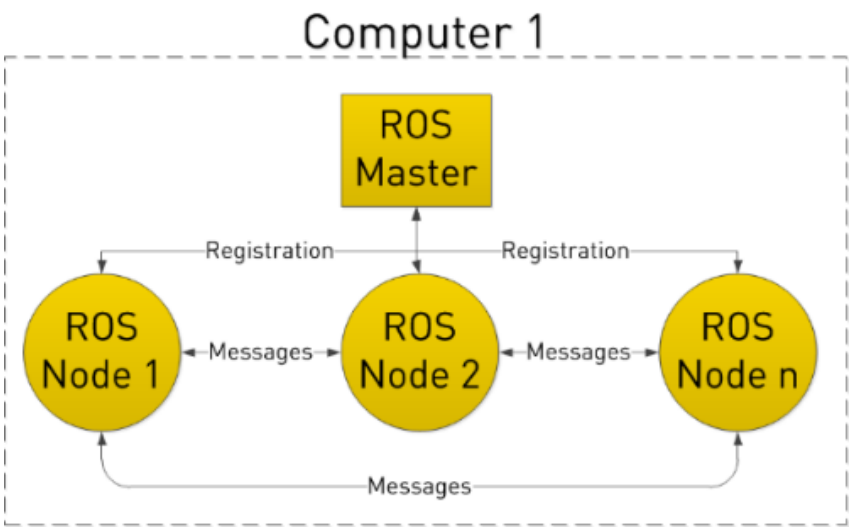

Figure 2: Observing the ROS Master

A node does this by publishing and subscribing to Topics. For example, we have a camera on our robot, and we want to be able to see the images from the camera, both on the Robot itself, and on another laptop. In this example, as shown in Figure 3, we have (i) a Camera Node that takes care of communication with the camera, (ii) an Image Processing Node on the robot that processes the image data, and (iii) an Image Display Node that displays images on the laptop screen. To start with, all Nodes have registered with the Master. Think of the Master as a platform where all the Nodes go to find where exactly to send messages.

Copyright 2026
No Unauthorized Duplication Permitted

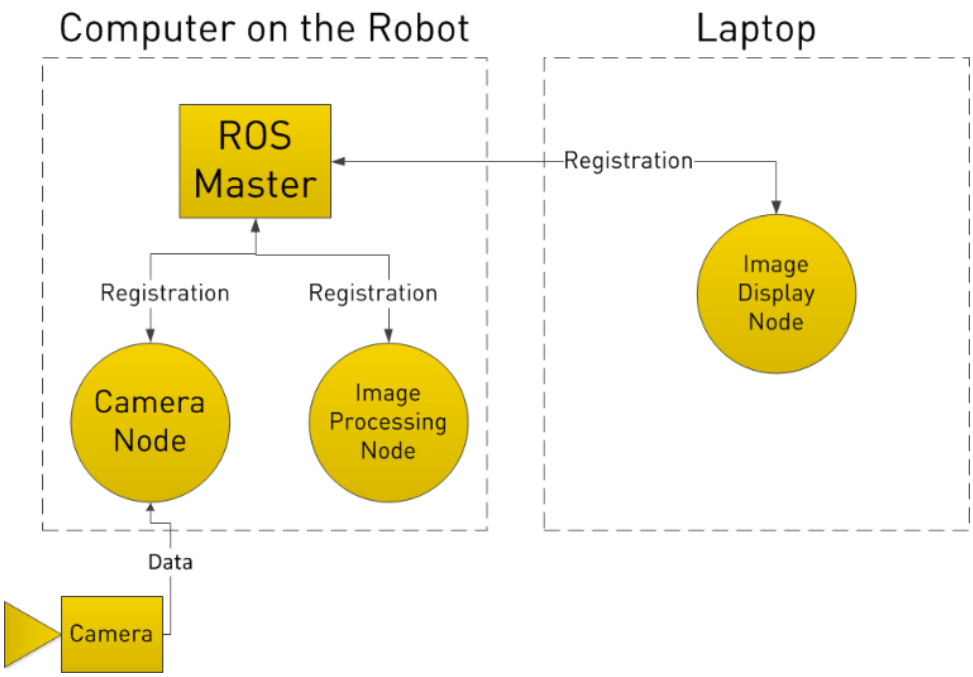

Figure 3: ROS example for viewing images on both the robot and on a laptop simultaneously

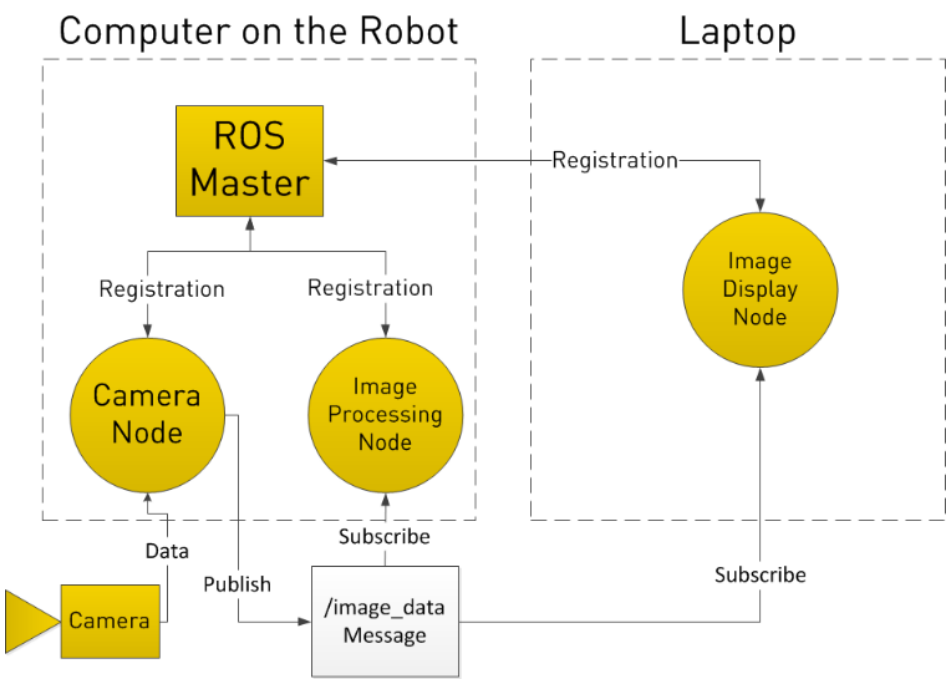

Figure 4: The Camera Node publishing a Topic called `/image_data`

Copyright 2026
No Unauthorized Duplication Permitted

In registering with the ROS Master, the Camera Node states that it will publish a *Topic* called `/image_data`, for example. Both the Image Processing Node and Image Display Node in Figure 4 register that they are Subscribed to the Topic `/image_data`. Thus, once the Camera Node receives some data from the camera hardware, it sends the `/image_data` message directly to the other two nodes.

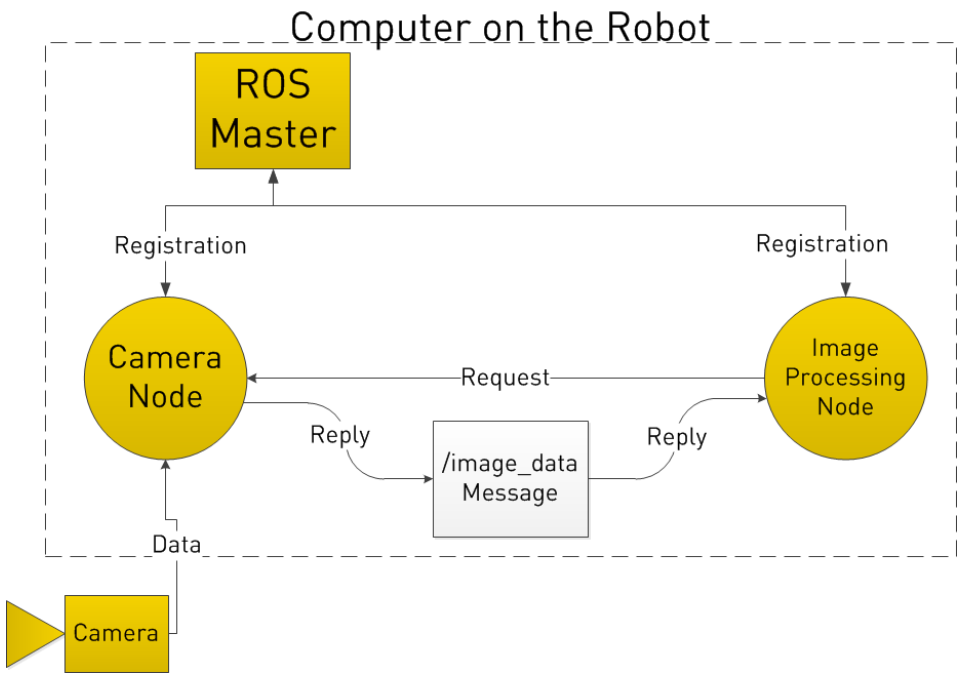

Figure 5: Requesting data from the camera node at a specific time.

Now you might be thinking, what happens if you want the Image Processing Node to request data from the Camera Node at a specific time. To do this, ROS implements **Services**. A **Node** can register a specific **Service** with the ROS Master, just as it registers its messages. In the example in Figure 5, the Image Processing Node first requests `/image_data`, the Camera Node gathers data from the camera hardware, and then sends the reply.

Copyright 2026
No Unauthorized Duplication Permitted

Frequently used Expressions and Commands for ROS:

Catkin: `Catkin` is the build system used for ROS. This means that after you have written all your code and defined your message files, you will run `catkin` to join everything together and compile any dependencies you have. To build packages, you must have a catkin workspace where all your projects are contained.

A `catkin` workspace or package can be built by running either of these two commands in the root of your catkin workspace: `catkin_make` or `catkin build`. Please note that the command you used for building a workspace should be consistent every time. For example, if the workspace was built using `catkin_make` the very first time while initializing the workspace, all the subsequent builds should be done using `catkin_make`. To change the method of build in a pre-existing workspace, you can use the command `catkin clean` which will clear everything except the source space (*src* folder) of the workspace. It can then be rebuilt using the desired command.

Workspace: A ROS workspace is basically a directory with a certain hierarchy wherein all your sets of related ROS codes are stored. A workspace contains packages which, in turn, contain the code for nodes amongst other things. You can have multiple workspaces on your computer but work in only one workspace at a time.

ROS Package: All the codes and software in ROS is organized in the form of packages. A package may contain ROS Nodes, libraries used in those codes, messages and service definitions, and all the dependencies. Once you have created your ROS package, you have to build it so that you can use it. The directory *src* is the place where the code for ROS nodes of the package is located.

ROS Nodes and Packages: A node is an executable file, such as written in python or C++, which performs a specific task and communicates with other nodes through topics. The communication between nodes is established mainly in two ways: (i) through publisher-subscriber relationship or (ii) through services. All the programming involved in creating a ROS based project is usually related to creating the nodes.

Copyright 2026
No Unauthorized Duplication Permitted

Publisher - Subscriber Model: This is a model through which two nodes can exchange data in the form of asynchronous messaging. It is usually used when a one-way stream of information is involved. For example, suppose you are developing a self-driving car and you have a node which processes the video feed from a camera to detect street signs. Whenever the node detects a sign, it needs to communicate type and distance of the sign from the car to another node which will use this information to control the speed of the car. In this scenario, your sign detection node is a publisher that publishes data about the signs it detects to a specific topic. We can call this topic as **signs**. Your controller node is a subscriber that subscribes to the **signs topic** and performs a certain task whenever it receives a message on the **signs topic**.

- **Publisher:** The publisher object of a ROS node publishes the data in the form of messages over a topic. There is a particular rate set by the user at which the messages get published.
- **Subscriber:** The data published by the publisher of one ROS node can be received or subscribed through the Subscriber object of a ROS node. The Subscriber object subscribes to the topic over which any publisher publishes the messages. Every Subscriber object is associated with a callback function which gets called every time a message is published over the topic.

Services: Another common mode of communication, especially suited when there is a transaction style relationship between the two nodes is the service. It consists of a *server* and a *client*. For example, consider in your self driving car, you have one node to control the movement of the car and another node to plan an optimal route to your destination. In such a scenario, the main control node is the *client* and the route planning node is the *server*. Suppose the controller node (*client*) finds out the route of the vehicle is blocked up ahead and the *client* requires a new route. Then, the *client* sends a request to the *server*. This request would consist of the current location, the destination, as well as any preferences the controller might have, such as a faster or a more comfortable route. The *server* would then compute an optimal route and return it to the *client* as the response.

Copyright 2026
No Unauthorized Duplication Permitted

- **Client:** This is the node which sends a request to a *server*. In the *client* code, you can create a special function which acts as a proxy between the *client* and *server*. Whenever you call that function, it sends a request to the server with the arguments passed to it as parameters.
- **Server:** This node constantly monitors for requests from the *client*. Whenever it receives one, it carries out the required task independently of the *client* and sends back the response when the task is complete.

For the success of your course project, it is essential that you learn and practice ROS. Here are a few links to study ROS in more details. You <u>must</u> practice the information given in these links.

Links to the ROS tutorial on ROS nodes:
http://wiki.ros.org/Nodes
http://wiki.ros.org/rosnode

Links to the ROS tutorial on ROS Topics:
http://wiki.ros.org/Topics
http://wiki.ros.org/rostopic

Links to the ROS tutorial on services:
http://wiki.ros.org/Services
http://wiki.ros.org/rosservice

Links to user guide on *rviz*:
http://wiki.ros.org/rviz/UserGuide

Please refer the ROS cheat sheet on Avenue that summarizes all the ROS concepts and commands.
<u>You must study the content and links given in ROS cheat sheet.</u>

Copyright 2026
No Unauthorized Duplication Permitted

Objective 2: Connecting to Jetson Nano

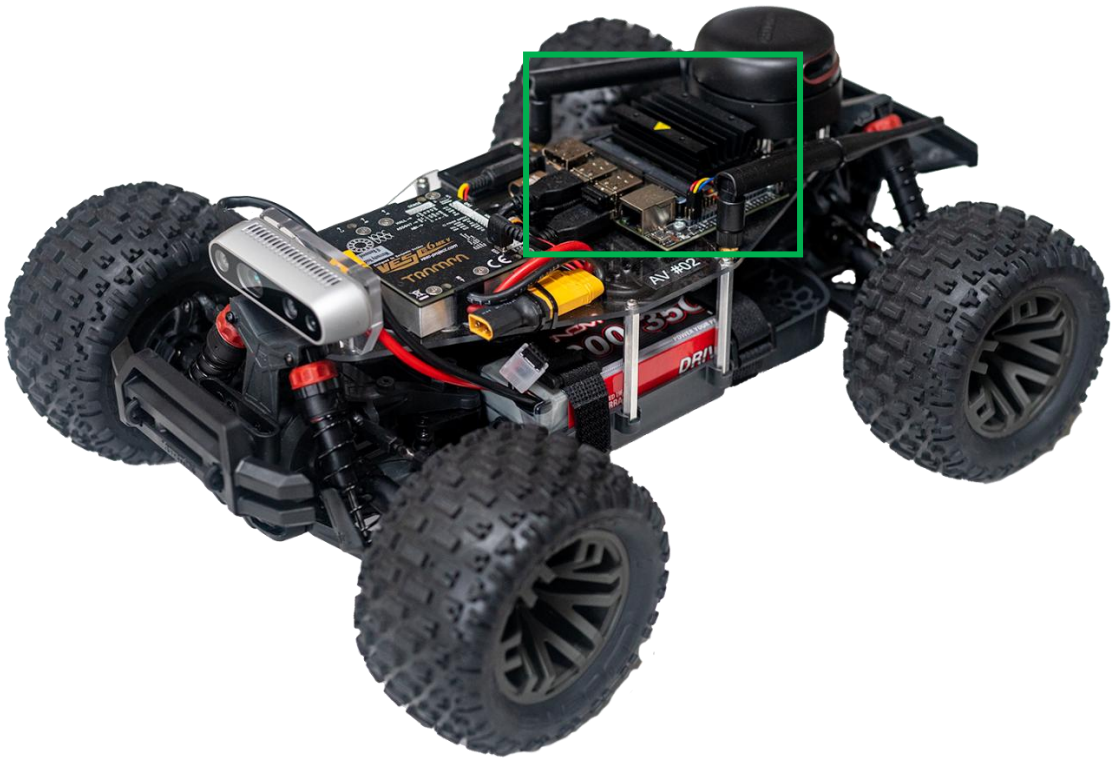

Figure 6: McMaster AEV and Jetson Nano

In this objective, you will connect to Jetson Nano, which is the main computer of the McMaster AEV. Jetson Nano and ROS are already set up in your vehicle kit. Figure 6 shows the location of Jetson Nano on the McMaster AEV. **Power the Jetson Nano with the portable switched-mode power supply** in your AEV kit**.** Plug the power supply into the round barrel jack on the Jetson Nano module (see Figure 7). At this point, **connect Jetson Nano to a monitor via an HDMI cable**. You should also **insert a wired mouse and a keyboard** into USB ports shown in Figure 7. If there are not enough ports, please unplug the VESC from Jetson Nano. After a few seconds, you should see the NVIDIA logo appear on the monitor and, after a few moments, you will be taken to the setup screen. Once you log in, you should see the Jetson Nano desktop as shown in Figure 8.

Default username: caruser

Default password: ELECENG 3EY4

Copyright 2026
No Unauthorized Duplication Permitted

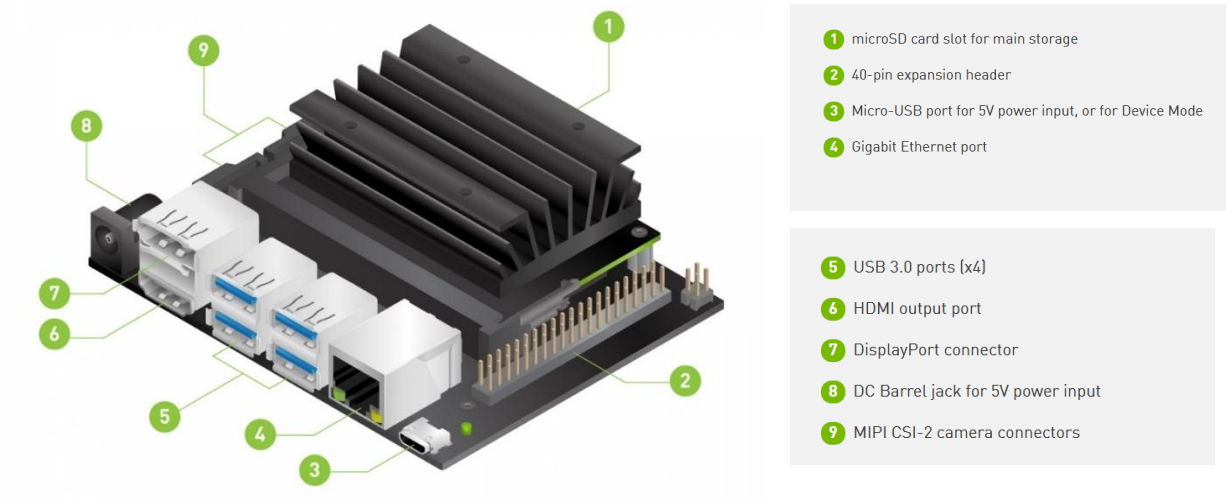

Figure 7: Diagram of Jetson Nano Developer Kit with important components labeled

Figure 8: Jetson Nano desktop

Copyright 2026
No Unauthorized Duplication Permitted

Objective 3: ROS Basics Practical Example

Open a new Terminal Window (Ctrl + Alt + T). In the new terminal, type roscore and press enter. This should produce something like in Figure 9. `roscore` command starts the ROS Master.

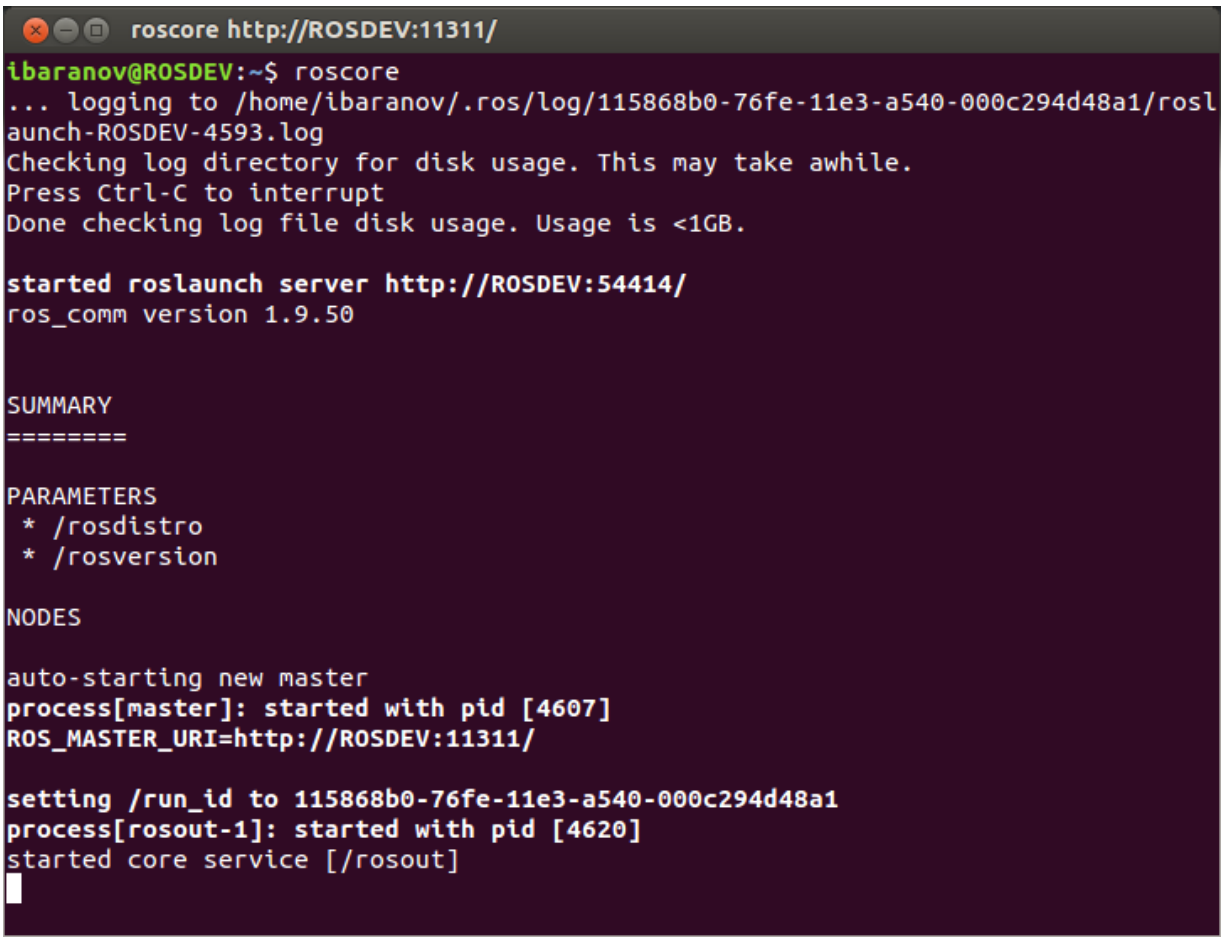

Figure 9: The result of `roscore` command (Do not close this terminal)

Open a new Terminal, and type in `rostopic`. This will give you a list of all the options that the `rostopic` command can do.

For now, we are interested in `rostopic` list. Type in `rostopic list` and press enter. This should give you a window as in Figure 10.

Copyright 2026
No Unauthorized Duplication Permitted

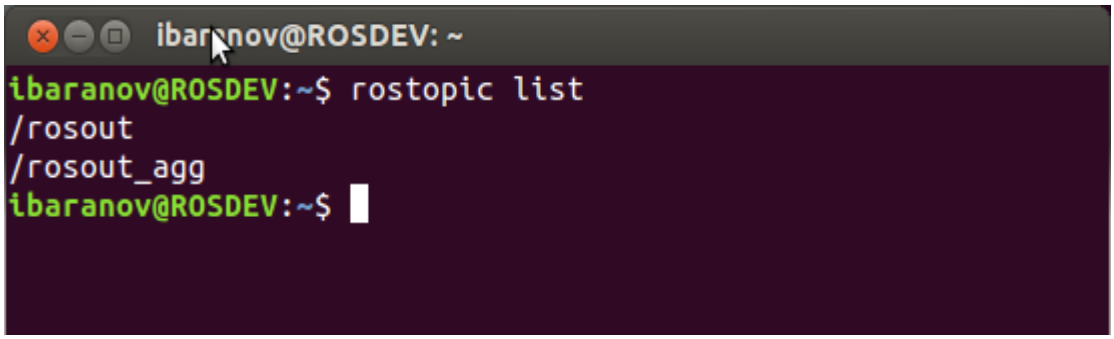

Figure 10: `rostopic` list command

<u>Question 1:</u> What is the purpose of the entries `/rosout` and `/rosout_agg` listed in Figure 10? Explain in detail.

What we want to do is to publish and subscribe to a message.

You can open a new **Terminal** again or open a new tab on the same terminal window (Ctrl + Shift + T). In the new Terminal, type in the following command as shown in Figure 11.

```
rostopic pub /hello std_msgs/String “Hello Robot"
```

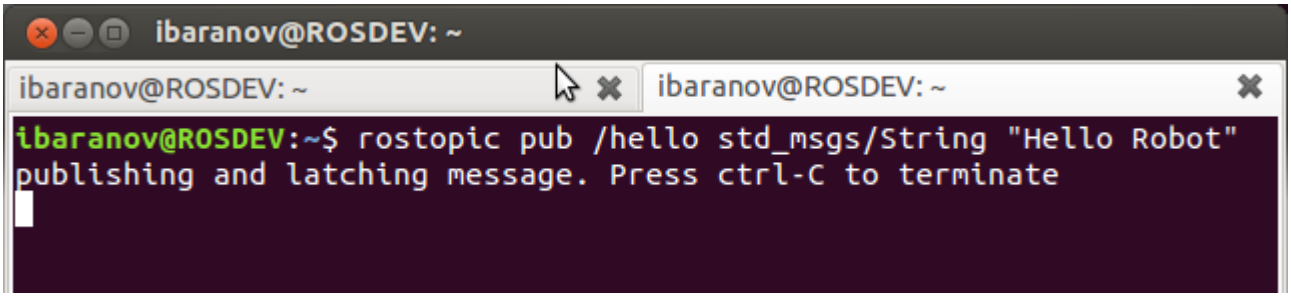

Figure 11: Publishing a message to a topic

Copyright 2026
No Unauthorized Duplication Permitted

Question 2: Break down the parts of the command in Figure 11 and explain each function: `rostopic pub, /hello, std_msgs/String, "Hello Robot".`

Now go back to the previous terminal where you executed `rostopic` command. Then, execute `rostopic list` again as shown in Figure 12. We now have a new topic listed! We can also echo the topic to see the message by entering `rostopic echo /hello`.

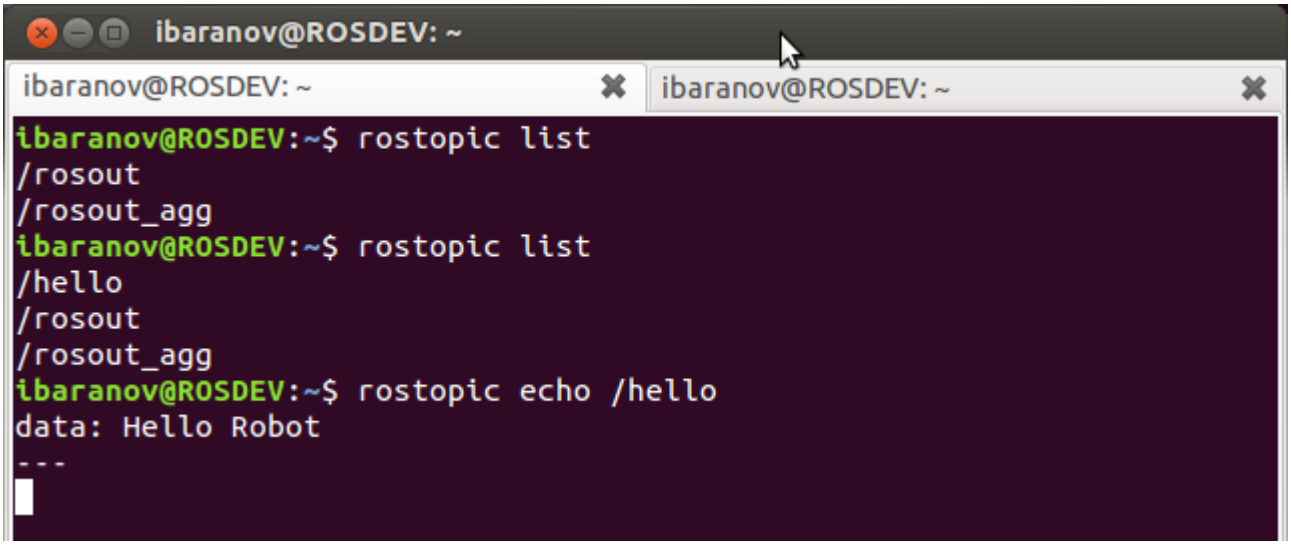

Figure 12: `rostopic` echo to display messages published to a topic

We have now successfully published a topic with a message and received that message. Type **Ctrl + C** to stop echoing the `/hello` topic.

Copyright 2026
No Unauthorized Duplication Permitted

We can also investigate the node that is publishing the message. Type in `rosnode list`. You will get a list like the one shown in Figure 13. The exact numbers beside the `rostopic` node may be different.

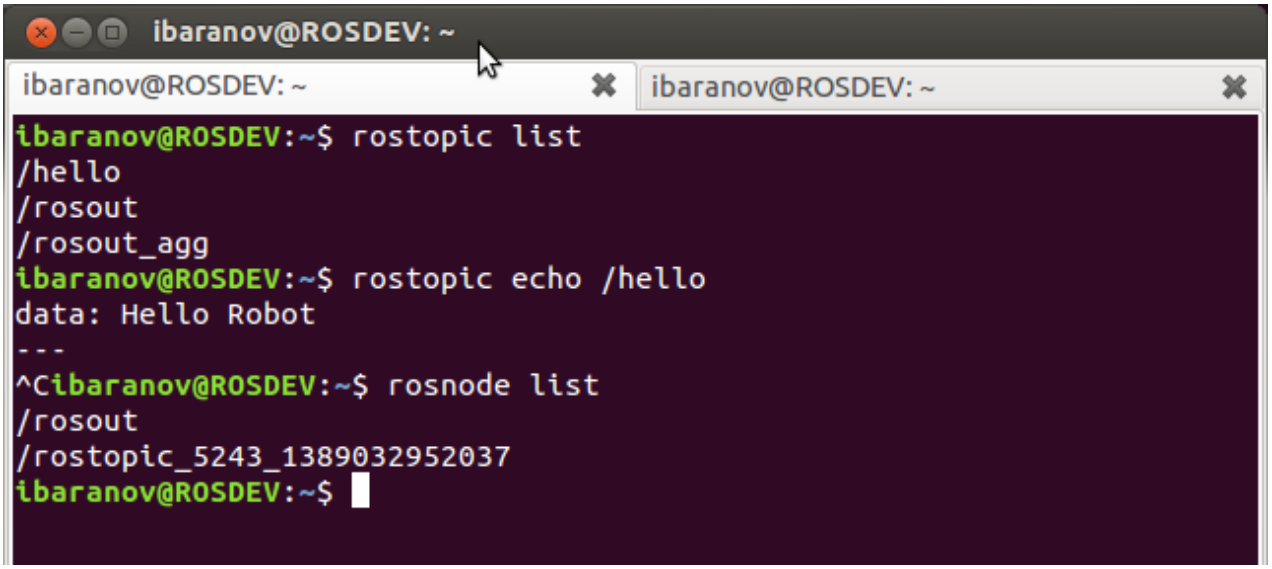

Figure 13: Using `rosnode` list to display a list of current nodes

Because we asked `rostopic` to publish the `/hello` topic for us, ROS went ahead and created a node to do so. We can look into the details of the node by typing `rosnode info /rostopic_….` **(the numbers that were assigned to the node in your system)**, as shown in Figure 14. It is fine if you do not get the Connections shown in Figure 14 in your terminal.

TIP: In ROS, and in Linux in general, whenever you start typing something, you can press the Tab key to auto complete it. If there is more than one entry, double tap Tab button to get the list. In the above example, all you must type is `rosnode info /rost` (TAB).

Copyright 2026
No Unauthorized Duplication Permitted

```
ibaranov@ROSDEV: ~
ibaranov@ROSDEV: ~    ibaranov@ROSDEV: ~
^Cibaranov@ROSDEV:~$ rosnode info /rostopic_5626_1389034143782
--------------------------------------------------------------------------------
----------
Node [/rostopic_5626_1389034143782]
Publications:
 * /hello [std_msgs/String]

Subscriptions: None

Services:
 * /rostopic_5626_1389034143782/get_loggers
 * /rostopic_5626_1389034143782/set_logger_level


contacting node http://ROSDEV:41234/ ...
Pid: 5626
Connections:
 * topic: /hello
    * to: /rostopic_5659_1389034166639
    * direction: outbound
    * transport: TCPROS

ibaranov@ROSDEV:~$
```

Figure 14: `rosnode info` command displaying information about a node, including publications and subscriptions

We can get information for our topic the same way, by typing `rostopic info /hello` as shown in Figure 15. You will notice that the node listed under "Publishers:" in Figure 15 is the same node we requested info about.

Copyright 2026
No Unauthorized Duplication Permitted

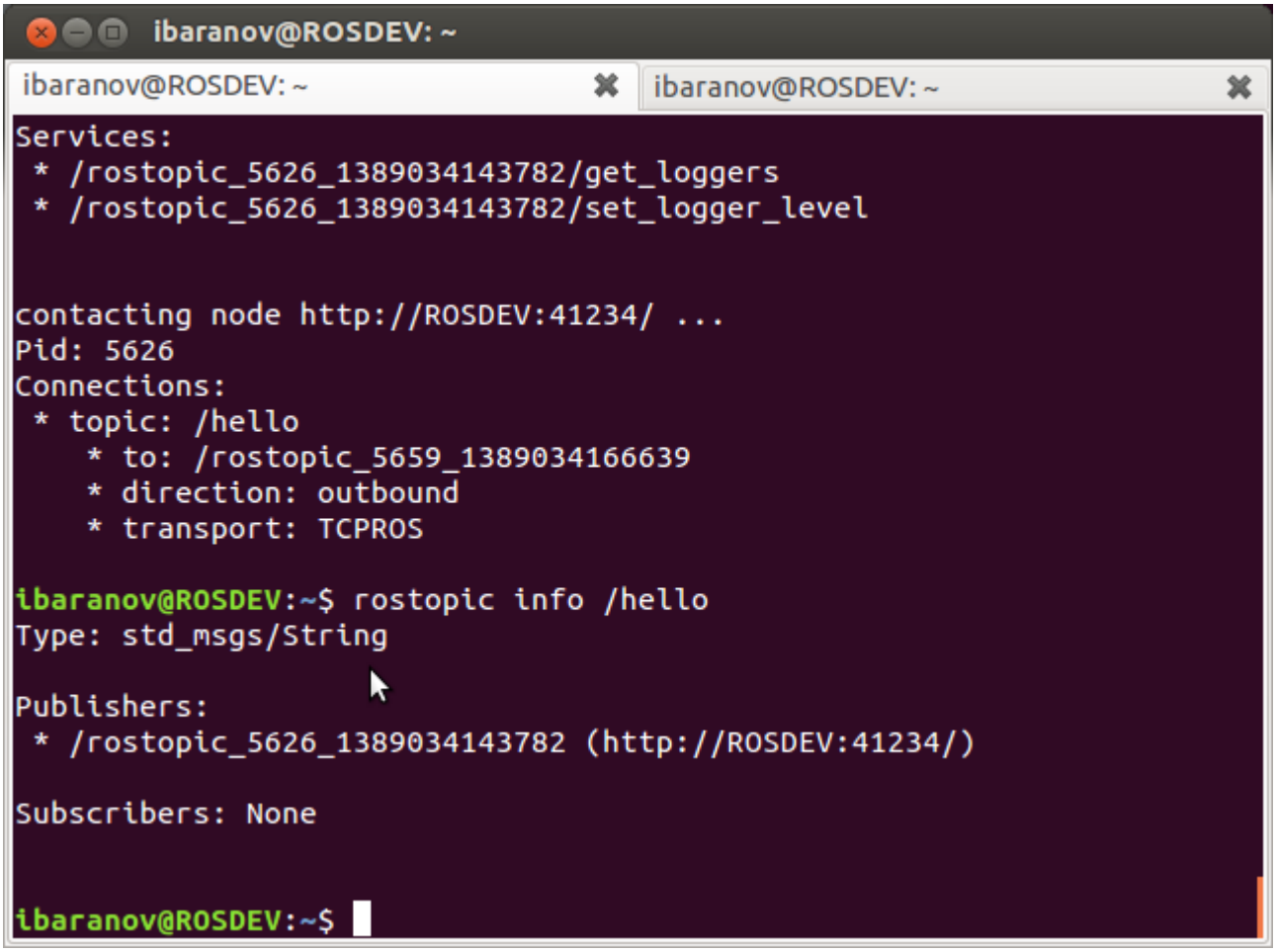

Figure 15: `rostopic info` command printing information about a topic, including publications and subscriptions

<u>Question 3:</u> Write and explain in your own words a summary of what you did in this objective and what you learned from it. Include screenshots of the output of the `rosnode info /rostopic/_…` command and of the `rostopic info /hello` command.

Copyright 2026
No Unauthorized Duplication Permitted

Objective 4: Setting up udev Rules

The udev rules are a set of files that specify an outside device that may be connected to a peripheral of Jetson Nano. The udev rules are used to identify devices that have been connected to Jetson Nano based on vendor ID or device ID. You need to add udev rules for all important components on the McMaster AEV, namely the VESC, lidar, and the camera. This is important because udev rules allow the Jetson Nano to detect devices and gain permissions to communicate with them on the start-up.

All udev rules can be found at `/etc/udev/rules.d`. You need to add three more, for lidar, VESC, and camera.

You will manually add three rule files, `rplidar.rules`, `vesc.rules` and `joypad-f710.rules`. For a better editing experience, you can use `gedit` instead of `nano`.

In this file, you will write line below (all in one line). After each comma, there is a space before the next letter. **Do not copy and paste the line; you must type it:**

```
KERNEL=="ttyUSB*", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="ea60", MODE:="0777", SYMLINK+="sensors/rplidar"
```

This tells Jetson Nano that, if a device with a vendor ID of 10C4 and a product ID of EA60 is using `ttyUSB*` to communicate, then the read, write, and execute permissions for it will be granted to all users, and a symbolic link to the device with the name `sensors/rplidar` will be created.

Use `lsusb` to check what is the actual vendor and product ID for the lidar. Please see the screenshot in Figure 16. Different devices may have different IDs.

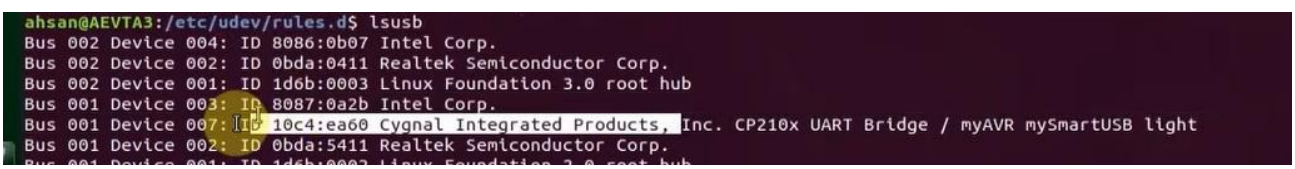

Figure 16: LIDAR vendor & Product ID highlighted.

Add the following to this file (all in one line). After each comma, there is a space before the next letter. **Do not copy and paste the line, you must type it:**

```
KERNEL=="ttyACM[0-9]*", ACTION=="add", ATTRS{idVendor}=="0483", ATTRS{idProduct}=="5740",
MODE="0666", GROUP="dialout", SYMLINK+="sensors/vesc"
```

Save and close the file.

Copyright 2026
No Unauthorized Duplication Permitted

Add the following to this file (all in one line). **Do not copy and paste the line, you must type it:**

```
KERNEL=="js[0-9]*", ACTION=="add", ATTRS{idVendor}=="046d", ATTRS{idProduct}=="c21f",
SYMLINK+="input/js0"
```

Copy the content given here and paste it in the new file. Then save the file and close it.

Now you can trigger these new rules by the executing the commands:

Caution: If you change a udev rule for any reason in the future, you must execute these commands after making your changes and saving the files.

```
$ sudo udevadm control --reload-rules
```

```
$ sudo udevadm trigger
```

Question 4: In your report, explain why you need to apply these commands and what they mean.

Lastly, you need to edit the ROS launch files for the lidar. The command `roscd` takes you directly to the location of a selected package. In each package, there is a launch folder containing launch files. Each launch file can be called in a single command to initialize a group of nodes, as well as specify any initial parameters.

First, navigate to the launch file:

```
$ roscd rplidar_ros
```

```
$ cd launch
```

```
$ sudo gedit rplidar.launch
```

Change the `value` in the "`serial_port`" line to "`/dev/sensors/rplidar`". Save and close.

Copyright 2026
No Unauthorized Duplication Permitted

Objective 5: Experiment with the data input and output of the Lidar

The experiments with the data input and output of Lidar might require powering up the vehicle with the battery. Therefore, these experiments should be conducted in the lab. Do not conduct the experiments related to Lidar using the power supply that powers up Jetson Nano only.

In next objectives, you will experiment with the data input and output with some of the devices onboard McMaster AEV. For this purpose, you will need ROS nodes, topics, and services, which you practiced previously. A ROS node is an executable that controls devices or performs computations. You will have many ROS nodes in your project. Topics and services are ways for nodes to communicate messages (data like lidar points and camera RGB values) between them. For example, the driver node for the lidar will send its data output via a topic to a node that handles calculation for servo (steering) motor position. This node will then output its calculation result via another topic to the driver node for the servo motor itself. This will enable the car to change its direction, thus avoiding obstacles.

You must study the following links before experimenting with ROS nodes and topics in this lab.

Link to the ROS tutorial on ROS Nodes: http://wiki.ros.org/ROS/Tutorials/UnderstandingNodes

Link to the ROS tutorial on ROS Topics: http://wiki.ros.org/ROS/Tutorials/UnderstandingTopics

Link to user guide on rviz:
http://wiki.ros.org/rviz/UserGuide

Open a new terminal and type in the following command:

```
$ ls /dev/sensors
```

Check if you see the file `rplidar`. Unplug the lidar from the Jetson Nano and run the command again. You should see that the file `rplidar` disappeared. If `rplidar` file is not available, you need to set up the package for the lidar. Check the appendix at the end of this manual for setting up the udev rules.

Copyright 2026
No Unauthorized Duplication Permitted

Make sure the Lidar is plugged back into the AEV before proceeding

Next, you will launch the lidar node with the following command:

```
$ roslaunch rplidar_ros rplidar.launch
```

Now, open a new terminal and run the following:

```
$ rosnode list
```

You should see `rplidarNode` under `rosnode`. Run the command `rosnode info rplidarNode`. You should see a topic called `/scan` being published by the node.

Next, run the following command:

```
$ rostopic list
```

You should see the same topic you saw earlier listed here. Use the command `rostopic info` on this topic, and you should see that this is being published by `rplidarNode`.

In the `type` section, you should see `sensor_msgs/LaserScan`. This is the message being conveyed over the topic published by the `rplidarNode` node. Use the following command to learn more about it:

```
$ rosmsg info LaserScan
```

You should see a sub-message called `ranges`, which is an array of floating numbers. This is the distance measurement obtained by the lidar around it. There are 720 data points in it, which represent the distance at 0 degrees to 359 degrees with 0.5 degree increments in the counter clockwise direction. To observe these data, run the following command:

```
$ rostopic echo scan
```

Use `ctrl+c` to stop the reporting of the data.

You should be able to find the array `ranges` there amongst the data dumps. To observe a single data point, for example, the <u>front of lidar</u>, use the following command:

```
$ rostopic echo scan/ranges[0]
```

For the <u>back of lidar</u>, use the following command:

Copyright 2026
No Unauthorized Duplication Permitted

```
$ rostopic echo scan/ranges[360]
```

Question 5: Put an object such as a notebook or a cardboard **at 270 degree around the lidar** (at least 20 cm away), move the object back and forth and explain how and why the numbers change. Add screenshots of the array ranges when the object is the closest to the car and when it is the furthest from the car.

Go back to the terminal where you initialized the lidar node and terminate it by pressing `ctrl+c`. Next, run the following command:

```
$ roslaunch rplidar_ros view_rplidar.launch
```

This will start both the lidar as well as `rviz`, which is a visualization tool for various raw data captured by all sorts of sensors. When the program opens, you should see a point cloud map of the surroundings of the lidar.

Question 6: Take a screenshot of the `rviz` page showing the point cloud map of the surrounding of the lidar and explain your observations of what the screenshot shows. Validate your observations by moving objects around the lidar and observe the change in the point cloud map.

To exit, go back to the terminal where you initialized this and click `ctrl+c`.

Copyright 2026
No Unauthorized Duplication Permitted

Objective 6: Experiment with the data input and output of the camera

Verify that Jetson Nano can detect the camera by using `lsusb`. See the screenshot in Figure 17 for how to check the camera vendor and product ID.

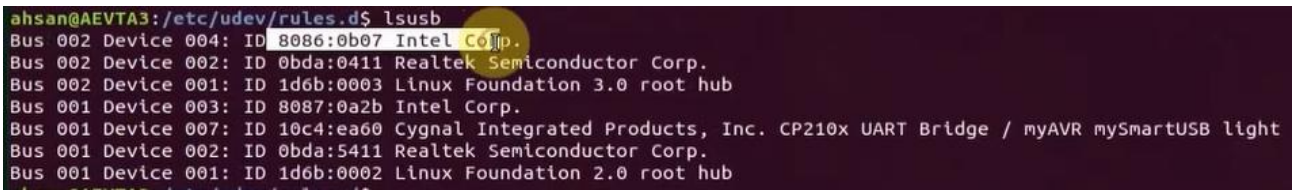

Figure 17: Camera vendor & Product ID highlighted

Once you have verified, simply run the following command to get the camera nodes up and running:

```
$ roslaunch realsense2_camera rs_camera.launch
```

If you see errors such as:

```
23/06 14:46:56,136 WARNING [547837067648] (messenger-libusb.cpp:42) control_transfer returned error, index: 768, error: No data available, number: 61
```

this is fine if you are still receiving data in the terminal window. You can check which topics are being published by the camera using `rostopic list`. Two of the important topics are `/camera/color/image_raw` and `/camera/depth/image_rect_raw`. By using `rostopic info` on these two topics, you can see that both are of `sensor_msgs/Image` type messages. These messages contain a massive array of 8-bit unsigned integers that represent the RGB values of pixels. Check that the Jetson Nano is receiving this data using `rostopic echo` for both topics (an 8-bit unsigned integer ranges from 0 to 255).

<u>Question 7:</u> In your report, include a screenshot of the output with the 8-bit unsigned integer range from 0 to 255 as you echo `/camera/color/image_raw` and `/camera/depth/image_rect_raw` topics.

Copyright 2026
No Unauthorized Duplication Permitted

You can also visualize this data using `rviz`. Open a new terminal and enter the following command:

```
$ rosrun rviz rviz
```

In the "fixed frame" section, select "camera_link", and then click "add" and add a camera display, as shown in Figure 18.

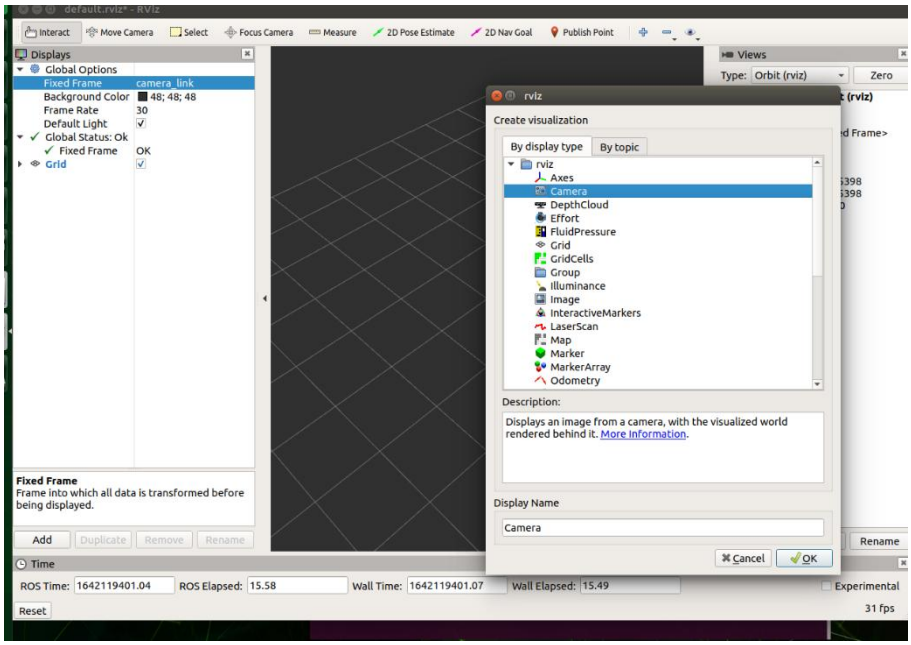

Figure 18: `rviz` interface

Add the topic received by this display to either `/camera/color/image_raw` and `/camera/depth/image_rect_raw`. You should be able to see a real-time camera feedback on the rviz interface as shown in Figure 19.

Copyright 2026
No Unauthorized Duplication Permitted

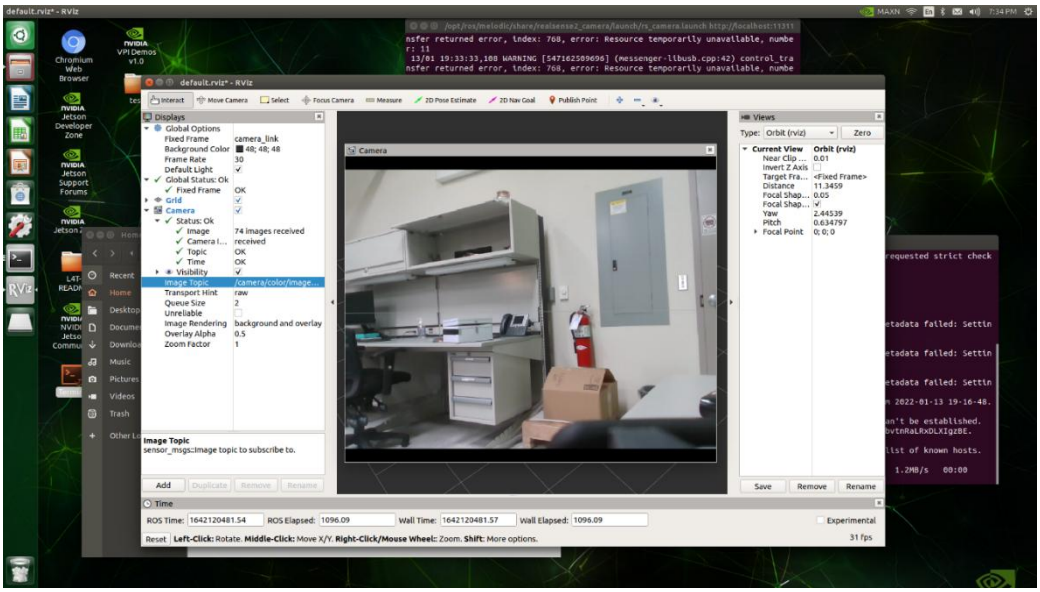

Figure 19: `rviz` real-time camera feedback

<u>Question 8:</u> Take a screenshot of the camera and depth camera simultaneously in one screenshot (Hint: you can do that by adding another camera page and selecting the topic you need). Explain briefly how the depth camera represents the raw coloured image. Also, take a screenshot of the raw coloured image showing ALL your team members.

<u>Objective 7: Experiment with the data input and output of the joystick</u>

You must review the manual of the Logitech F710 controller before you work on this objective: https://www.logitech.com/assets/34885/3/f710620-002923003403gswamr.pdf

Plug in the dongle into a USB port of a Jetson Nano and to check if the connection is successful. Use the `cd` command to navigate to `/dev/input` and verify that a folder called `js0` is present by typing `ls`. To confirm that the controller is powered on, press the Vibration button and see if the controller vibrates. **Make sure that, on the joystick controller, the input toggle is switched to "X" (for Xinput) and the mode LED is off**. When you execute `lsusb` command, the product name for the joystick should have [Xinput Mode]. To activate the joystick driver, first navigate to the launch folder of the joystick driver package:

Copyright 2026
No Unauthorized Duplication Permitted

```
$ cd ~/catkin_ws/src/joystick_drivers/joy/launch
```

You will create a custom launch file for the joystick. Use the following command:

```
$ sudo gedit f710joy.launch
```

Then, **type** the following content into the launch file. **Do not copy and paste the content**.

```
<launch>
  <node name="joy_node" pkg="joy" type="joy_node">
    <param name = "dev" value = "/dev/input/js0"/>
    <param name = "autorepeat_rate" value = "2" />
    <param name = "coalesce_interval" value = "0.05" />
  </node>
</launch>
```

`coalesce_interval` parameter defines how long it takes until the next set of message is published. It can help improve the response time of the vehicle to the input command. Save the file and close. This launch file recognizes `js0` as the controller and set the autorepeat rate as 2Hz. This means that, each second, two new sets of messages are transmitted if no toggle position is changed.

Question 9: The above explanation should have given you an idea about the parameters you added to the launch file. In your report, explain each line that you typed into the launch file (e.g., the four lines in the middle).

To activate the joystick node, simply run the command below.

```
$ roslaunch joy f710joy.launch
```

If you receive an error saying , `Couldn't set gain on joystick force feedback: bad file descriptor`, it should be fine. You can move forward.

Open another terminal. If you use `rosnode info` on `joy_node`, you can see that one of the topics that this node publishes is called `/joy`. Using the command `rostopic info` on `/joy`, you will see that the message over this topic is `sensor_msgs/Joy`.

Copyright 2026
No Unauthorized Duplication Permitted

Next, use `rostopic echo` on `/joy`, you can see that the messages `sensor_msgs/Joy` consists of two arrays, one called `axes` and one called `buttons`. By pressing the buttons and pulling the controller sticks, you can see that the number changes accordingly.

Question 10: Find out which element in the two arrays, `axes` and `buttons` correspond to which button by trying them all and seeing how the numbers change. Add screenshots showing the two arrays when each button is pressed and tested.

Important note: **You must remove the batteries from your joystick at the end of this lab, before placing it back in your suitcase. Also, please make sure the batteries are removed when not in use in future labs and when stored in your suitcase**. This is because the joystick has an auto-turn on function when you press on any of the buttons. Since, the joystick is stored in the case of your McMaster AEV, it can easily touch any surface, which activate the buttons and turn on the controller when not in use. This will affect the battery life.